AgroScan — API, Backend e Banco de Dados

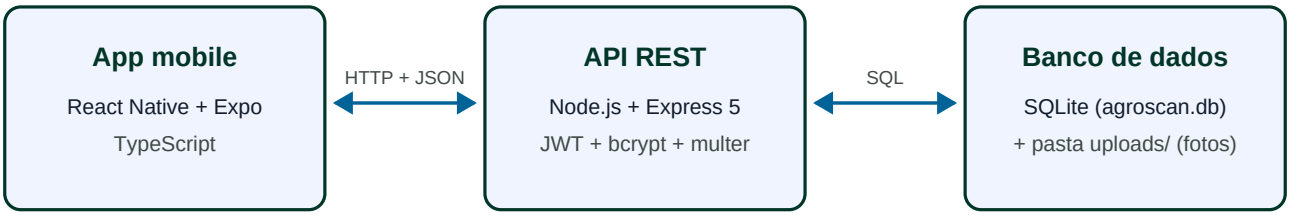
Guia de estudo para a arguição do professor: o que foi feito, como funciona e como explicar cada parte.

> **Resposta de 30 segundos.** O app (React Native/Expo) não guarda dados sozinho: ele conversa por HTTP em JSON com uma **API REST em Node.js + Express**, que autentica com **JWT**, guarda senhas com **bcrypt** e persiste tudo num **banco SQLite** (5 tabelas relacionadas). Fotos ficam numa pasta e o banco guarda só o caminho. Cada usuário só enxerga os próprios dados.

O que o professor anotou → onde está pronto

Anotação do professor	Como está no projeto	Como mostrar
Backend API (decidir; alternativa: JSON Server no N1)	Decidimos por um **backend real** (Express + SQLite), que é mais completo que o JSON Server: tem autenticação de verdade, regras no servidor, upload de arquivos e relacionamento entre tabelas.	Terminal: `npm run dev` e `curl localhost:4000/health`
Front (ok, com mapeamentos para depois do N1)	O front já consome a API. O **mapeamento** é feito em dois lados: o servidor converte colunas do banco (snake_case) para o JSON da API (camelCase) e o app tem tipos TypeScript que espelham esse JSON.	Abrir `src/api/client.ts` e `src/types/index.ts`
Para o N1: autenticação funcionando	Cadastro, login, JWT, rotas protegidas (401 sem token), sessão que persiste ao fechar o app e logout.	No app + `curl` sem token (401) e com token (200)
Banco de dados	SQLite, 5 tabelas com chaves estrangeiras e **ON DELETE CASCADE**; senhas só como hash.	`npm run db` mostra todas as tabelas

1. Arquitetura geral



O app **nunca** acessa o banco diretamente. Todo dado passa pela API, que valida, aplica as regras e só então lê/grava no banco. O servidor é **stateless**: não guarda sessão em memória; cada requisição traz o token que prova quem é o usuário.

Mapa de arquivos do backend (pasta `backend/`)

Arquivo	Responsabilidade
`src/index.js`	Monta o Express: **cors**, **express.json**, serve **/uploads** (fotos), registra as rotas, trata 404 e erro 500 e escuta em **0.0.0.0:4000** (todas as redes, senão o celular não alcança).
`src/db.js`	Abre o SQLite, ativa o modo WAL, cria as tabelas (se não existirem) e faz a migração das colunas de foto. Cria a pasta `data/` se ela não existir.
`src/seed.js`	Catálogo de 6 pragas (carregado no start) e dados de exemplo para cada conta nova (4 setores, 3 ordens, 3 detecções).
`src/middleware/auth.js`	`requireAuth`: lê o header Authorization, valida o JWT e coloca o id do usuário em `req.userId`.
`src/upload.js`	Configuração do **multer** (recebe fotos, limite de 8 MB, nome aleatório) e função que monta a URL absoluta da foto.
`src/routes/auth.js`	Cadastro, login, perfil, foto de perfil e capa da fazenda.
`src/routes/detections.js`	Histórico de análises e criação de uma nova (com a "IA" simulada).
`src/routes/orders.js`	Ordens de serviço: listar, criar e mudar status.
`src/routes/sectors.js`	Setores da fazenda do usuário.
`scripts/inspect-db.js`	`npm run db` — mostra o banco em tabelas.
`scripts/smoke.js`	`npm run smoke` — 51 verificações automáticas de ponta a ponta.

2. Tecnologias e por que cada uma

Tecnologia	Papel	Por que escolhemos / trade-off
Node.js	Executa JavaScript no servidor.	Mesma linguagem do app (JS/TS): um único idioma no projeto inteiro.
Express 5	Framework HTTP: rotas e middlewares.	Minimalista e o mais usado no ecossistema Node. Na v5, erros em funções async caem no tratador de erro automaticamente.
SQLite (better-sqlite3 12.11.1)	Banco relacional em **um arquivo** (`data/agroscan.db`).	Zero instalação (não precisa de servidor de banco) mas é SQL de verdade, com chaves e JOINs. **Limite:** um arquivo/um servidor; em produção migraríamos para PostgreSQL. A versão foi fixada porque a 13 tenta compilar C++.
bcryptjs	Faz o hash das senhas.	Hash lento + salt aleatório: mesmo com o banco vazado, senhas não são reversíveis. Implementação em JS puro (não precisa compilar).
jsonwebtoken	Cria e valida os tokens JWT.	Sessão sem estado no servidor. Algoritmo HS256, validade de 30 dias.
multer	Recebe uploads `multipart/form-data`.	Salva a foto em disco com nome aleatório (UUID) e limite de 8 MB.
cors	Libera requisições de outra origem.	Necessário para o app web/desenvolvimento. Está aberto (qualquer origem) porque é ambiente de demonstração.
dotenv	Lê variáveis do arquivo `.env`.	Configuração sem mexer no código: `PORT`, `JWT_SECRET`, `DB_PATH`.
nodemon	Reinicia o servidor ao salvar um arquivo.	Só em desenvolvimento (`npm run dev`).

3. Banco de dados

Por que relacional? Os dados têm relações claras (um usuário tem vários setores, ordens e análises; cada análise aponta para uma praga). O banco garante essas relações com chaves estrangeiras, coisa que um JSON solto não faz.

Modelo (relacionamentos 1 para N)

```
users (1) ----< sectors          (um usuário tem vários setores)
users (1) ----< detections       (um usuário tem várias análises/fotos)
users (1) ----< service_orders   (um usuário tem várias ordens de serviço)
pests (1) ----< detections       (uma praga do catálogo aparece em várias análises)

Todas as tabelas filhas têm user_id  ->  FOREIGN KEY ... REFERENCES users(id) ON DELETE CASCADE
```

Tabelas e colunas

Tabela	Colunas principais	Observações
users	id (INTEGER, chave primária, autoincremento) · name · email (**UNIQUE**) · password_hash · farm_name · created_at (ms) · avatar_url · farm_photo_url	Só existe o **hash** da senha. `avatar_url` e `farm_photo_url` foram adicionadas depois por **migração**.
pests	id (TEXT, ex: "ferrugem-asiatica") · name · scientific_name · crop · severity (critical/moderate/low) · description · recommended_action · affected_area_min/max	Catálogo **compartilhado** (6 pragas), não pertence a nenhum usuário.
sectors	id (TEXT) · user_id (FK) · name · crop · hectares · health_score · status (optimal/review/syncing)	Setores da fazenda de cada usuário.
detections	id (TEXT) · user_id (FK) · pest_id (FK para pests) · confidence · sector · timestamp · photo_url · affected_area · stage · offline (0/1)	Uma linha por análise feita no scanner. `photo_url` guarda só o caminho ("/uploads/xxx.jpg") e pode ser nulo.
service_orders	id (TEXT) · user_id (FK) · code (ex: OS-2026-106) · product · sector · crop · date · status (pending/in_progress/completed)	O `code` é sequencial por usuário (maior número existente + 1).

Conceitos que você precisa saber explicar

- **Chave primária (PK):** identifica cada linha de forma única. **Chave estrangeira (FK):** coluna que aponta para a PK de outra tabela (ex: `detections.user_id` → `users.id`) e impede apontar para algo que não existe. O SQLite do better-sqlite3 já vem com as FKs **ligadas**.
- **UNIQUE no e-mail:** o próprio banco impede duas contas iguais (além da checagem que a API faz antes e que devolve 409).
- **ON DELETE CASCADE:** apagar um usuário apaga automaticamente setores, análises e ordens dele. O `npm run smoke` usa isso para se limpar sozinho.
- **JOIN:** para devolver uma análise com os dados da praga, a API junta `detections` com `pests` numa só consulta e monta o objeto `pest` aninhado.
- **Prepared statements:** todas as consultas usam parâmetros `?` (ex: `WHERE email = ?`), nunca concatenam texto do usuário no SQL. Isso evita **SQL injection**.
- **Modo WAL** (`journal_mode = WAL`): leituras não bloqueiam escritas. É por isso que aparecem os arquivos `agroscan.db-wal` e `-shm` ao lado do banco.
- **Seed (dados iniciais):** as pragas entram com `INSERT OR IGNORE` (idempotente: pode rodar todo start sem duplicar). Cada cadastro novo recebe 4 setores, 3 ordens e 3 análises de exemplo para a demo não abrir vazia.
- **Migração:** como o banco já existia, as colunas de foto foram adicionadas com `PRAGMA table_info` + `ALTER TABLE ADD COLUMN`, só se ainda não existiam. Assim ninguém perde dados.

- **Ids como TEXT:** ordens e análises podem ser criadas no app/servidor com id gerado (timestamp + parte de UUID); só o usuário usa inteiro autoincremental.
- **Fotos:** o arquivo fica em `backend/uploads/` e o banco guarda o **caminho relativo**. A URL completa é montada na resposta com o host que o cliente usou, por isso as fotos continuam funcionando quando o IP do notebook muda.
- **Onde está o banco:** `backend/data/agroscan.db`. A pasta `data/` não vai para o GitHub (dados de usuários não devem ser versionados); é criada sozinha no primeiro start.

4. A API REST

REST = recursos com URLs (`/orders`, `/detections`) e verbos HTTP com significado: **GET** lê, **POST** cria, **PATCH** altera parte de um recurso. A resposta é sempre JSON: sucesso → `{ "orders": [...] }`; erro → `{ "error": "mensagem" }`.

Método	Rota	Token?	O que faz	Respostas
GET	/health	não	Confere se o servidor está no ar.	200
POST	/auth/register	não	Cria conta: valida, faz o hash da senha, cria dados de exemplo e devolve token + usuário.	201 · 400 sem e-mail/senha · 409 e-mail já existe
POST	/auth/login	não	Confere e-mail e senha e devolve token + usuário.	200 · 400 · 401 credenciais erradas
GET	/auth/me	sim	Dados do usuário logado (usado para validar a sessão ao abrir o app).	200 · 401
PATCH	/auth/me	sim	Edita nome e nome da fazenda.	200 · 401
POST	/auth/me/avatar	sim	Foto de perfil (multipart, campo "photo").	200 · 400 sem arquivo
POST	/auth/me/farm-photo	sim	Foto de capa da fazenda (multipart).	200 · 400 sem arquivo
GET	/sectors	sim	Setores do usuário.	200
GET	/detections	sim	Histórico de análises, mais recente primeiro, com a praga aninhada.	200
POST	/detections	sim	Recebe a foto, faz a análise (simulada) e grava.	201
GET	/orders	sim	Ordens de serviço do usuário.	200
POST	/orders	sim	Cria ordem (status inicial pending, código único).	201 · 400 campos faltando
PATCH	/orders/:id	sim	Muda o status (pending / in_progress / completed, nos dois sentidos).	200 · 400 status inválido · 404 não existe ou é de outro usuário
GET	/uploads/<arquivo>	não	Serve as fotos salvas (arquivos estáticos).	200

Códigos HTTP em uma linha cada

- **200 OK** deu certo · **201 Created** criou algo · **400 Bad Request** o cliente mandou dado inválido/faltando.
- **401 Unauthorized** não autenticado (sem token, token inválido/expirado, senha errada) · **404 Not Found** não existe · **409 Conflict** conflito (e-mail já cadastrado) · **500** erro interno do servidor.
- **Por que 404 e não 403 quando outro usuário tenta mexer na minha ordem?** Para não revelar que a ordem existe.

5. Autenticação (o ponto central do N1)

Cadastro e login passo a passo

1. **Cadastro** (`POST /auth/register`): o app envia e-mail, senha e nome da fazenda. O servidor normaliza o e-mail (sem espaços, minúsculo), confere se já existe (409), faz `bcrypt.hash(senha, 10)` e grava **só o hash**.
2. O servidor cria os dados de exemplo do usuário e gera o token: `jwt.sign({ sub: id }, SEGREDO, { expiresIn: '30d' })`. Responde com `{ token, user }`.
3. **Login** (`POST /auth/login`): busca o usuário pelo e-mail e usa `bcrypt.compare(senhaDigitada, hash)`. Usuário inexistente e senha errada devolvem a **mesma mensagem** ("E-mail ou senha incorretos") para não revelar quais e-mails existem.
4. O app guarda o token no **AsyncStorage** (chave `agroscan:token`) e o `api/client.ts` envia em toda requisição: `Authorization: Bearer <token>`.
5. **Rotas protegidas**: o middleware `requireAuth` verifica a assinatura e a validade (`jwt.verify`) e coloca o id em `req.userId`. Sem token ou token ruim → **401**.
6. **Isolamento entre usuários**: toda consulta filtra `WHERE user_id = req.userId`. Um usuário nunca lê nem altera dado de outro.
7. **Sessão que persiste**: ao abrir o app, se há token, ele chama `GET /auth/me`. Se for válido, entra direto; se falhar (expirou / servidor fora), volta para o login. **Logout** apenas apaga o token local (o JWT é stateless: o servidor não guarda nada).

Dois conceitos que caem sempre

JWT (JSON Web Token)	bcrypt
Três partes em base64 separadas por ponto: **cabeçalho . payload . assinatura**. O payload aqui é `{ sub: 2, iat, exp }`. **Não é criptografado, é assinado** (HMAC-SHA256 com o segredo): qualquer um consegue ler o payload (por isso nunca se coloca senha nele), mas ninguém consegue alterá-lo sem o segredo. Válido por 30 dias.	É um **hash** (não uma criptografia): não existe "descriptografar"; para conferir, recalcula-se e compara-se. Cada senha ganha um **salt** aleatório, então a mesma senha gera hashes diferentes. Guardado como `$2b$10$...`: **2b** = versão, **10** = custo (2^10 rodadas, propositalmente lento para dificultar força bruta), depois salt e hash.

Autenticação x autorização: autenticação = *quem é você* (login/token); autorização = *o que você pode fazer/ver* (o filtro por `user_id`).

6. Fluxos de ponta a ponta

Scanner (a foto até o banco)

1. A câmera gera um arquivo local (`file://...`). O app o embrulha na classe `File` do `expo-file-system` (que se comporta como um **Blob**) e monta um `FormData`.
2. `POST /detections` em `multipart/form-data`. O **multer** grava o arquivo em `backend/uploads/` com nome aleatório.
3. O servidor faz a "análise": sorteia uma praga do catálogo, gera confiança entre 60 e 99, área afetada dentro da faixa daquela praga e escolhe um setor do usuário. **Isso é simulado** (ver seção 9).
4. Grava em `detections` (com o `user_id` e o caminho da foto), faz o JOIN com `pests` e responde **201** com a análise completa e a URL da foto.
5. O app coloca a análise no histórico e abre o relatório. O que aparece na tela veio do servidor: **a fonte da verdade é o banco**.

Ordem de serviço

Criar (`POST /orders`) e mudar status (`PATCH /orders/:id`) só atualizam a tela **depois** que o servidor responde. O status pode ir e voltar (pendente ↔ em andamento ↔ concluída) e a API valida se o valor é um dos três permitidos.

7. Como o app conversa com a API (o "mapeamento")

- `src/api/client.ts`: função central `request()` que monta a URL, anexa o token, aplica **timeout de 15 s** e transforma falhas em `ApiError` com mensagem clara. Uma função por endpoint (login, fetchOrders, createDetection...).
- `src/context/AppContext.tsx`: estado global (usuário, histórico, ordens, setores). Cada ação chama a API e só então atualiza o estado.
- `src/types/index.ts`: interfaces TypeScript que espelham o JSON da API.
- **Mapeamento banco → API:** as colunas usam snake_case (`farm_name`, `health_score`, `photo_url`) e a API responde em camelCase (`farmName`, `healthScore`, `photoUri`). Cada rota tem uma função de conversão (`toPublicUser`, `rowToDetection`, `rowToOrder`, `rowToSector`). A senha nunca sai do servidor.
- **Endereço do servidor:** prioridade = endereço salvo pelo usuário > IP detectado (o mesmo do Metro, que roda no mesmo notebook) > padrão. A tela de login mostra "Servidor: ... Alterar" para corrigir sem precisar entrar.

8. Demonstração ao vivo (roteiro de ~3 minutos)

Use o **Git Bash** (no PowerShell `curl` é outro comando). Terminal 1 com o servidor ligado; os comandos abaixo no Terminal 2.

```
# Terminal 1
cd backend
npm run dev
```

1) Servidor no ar, 2) login guardando o token, 3) rota protegida com e sem token, 4) olhar dentro do JWT:

```
curl http://localhost:4000/health

TOKEN=$(curl -s -X POST http://localhost:4000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"teste@agroscan.com","password":"senha1234"}' \
  | grep -o '"token":"[^"]*' | cut -d'"' -f4)

curl -s http://localhost:4000/sectors -H "Authorization: Bearer $TOKEN"

curl -s -w "\nHTTP %{http_code}\n" http://localhost:4000/orders

node -e "console.log(JSON.parse(Buffer.from(process.argv[1].split('.')[1],'base64url')))" "$TOKEN"
```

5) Mostrar o banco e 6) rodar os testes automáticos (em `backend/`):

```
npm run db
npm run smoke
```

Sequência que impressiona

1. Mostrar o **401** sem token e o **200** com token: prova que a rota é protegida.
2. Decodificar o token e mostrar `sub` (id do usuário) e a validade de 30 dias (`exp - iat`).
3. No app, **criar uma conta nova**. Rodar `npm run db`: aparece a linha nova em **users** com **password_hash** começando com `$2b$10$` (a senha não está lá) e os 4 setores, 3 ordens e 3 análises criados para ela.

4. No app, **escanear uma foto**. Rodar `npm run db` de novo: nova linha em **detections** e um arquivo novo em `backend/uploads/`.
5. Fechar e abrir o app: continua logado (sessão persistente pelo token). Sair e entrar com a senha errada: mensagem de erro do servidor.
6. `npm run smoke`: 51 verificações passando; ele cria contas de teste e apaga tudo no final (cascade).

> **Antes de mostrar:** use uma conta sua (ou crie uma nova). A conta `teste@agroscan.com` tem fotos de perfil/capa de teste inválidas e várias ordens duplicadas de testes antigos.

9. Perguntas prováveis e respostas curtas

Por que não usou JSON Server (a alternativa que o professor liberou)?

O JSON Server só expõe um arquivo JSON como API; não tem autenticação nativa, hash de senha, upload de arquivo nem regras de negócio. Como o N1 exige autenticação funcionando e queríamos dados relacionados por usuário, construímos um backend real em Express com SQLite.

Por que SQLite? E em produção?

Porque não exige instalar um servidor de banco: é um arquivo, ideal para desenvolvimento e demonstração, e mesmo assim é SQL relacional completo. Em produção, com muitos usuários simultâneos, migraríamos para PostgreSQL; como usamos SQL padrão e todas as consultas estão nas rotas, a troca é localizada.

Como a senha é guardada? Dá para descobrir a senha de alguém olhando o banco?

Só o hash bcrypt é guardado (você pode ver com npm run db). Não existe função inversa: para validar o login, o servidor recalcula o hash da senha digitada usando o salt que já está no hash e compara. O custo 10 torna tentativas de força bruta lentas.

O que é JWT e como o servidor sabe quem é o usuário?

É um token assinado com um segredo que só o servidor conhece. Ele carrega o id do usuário (sub) e a data de validade. A cada requisição o app manda o token no header Authorization; o middleware verifica a assinatura e coloca o id em req.userId. Assim o servidor não precisa guardar sessão.

O que impede um usuário de ver ou alterar dados de outro?

Toda consulta filtra por user_id = req.userId, que vem do token (e não de algo que o cliente possa forjar). O smoke test verifica isso: o usuário 2 não vê nem consegue alterar a ordem do usuário 1 (recebe 404).

Onde o token fica no celular? É seguro?

No AsyncStorage, que não é criptografado. Para uma demonstração acadêmica serve; em produção usaríamos armazenamento seguro do sistema (Keychain/Keystore, via expo-secure-store) e tokens de curta duração com refresh token.

O que acontece se o token expirar ou for inválido?

A API responde 401. Ao abrir o app, a checagem de sessão (GET /auth/me) falha e o app volta para a tela de login.

O que é middleware?

Uma função que roda no meio do caminho da requisição, antes da rota. Usamos: cors (permite outras origens), express.json (lê o corpo JSON), multer (lê upload) e requireAuth (valida o token).

O que é CORS e por que está aberto?

É uma regra dos navegadores que bloqueia chamadas entre origens diferentes, a menos que o servidor autorize. Deixamos aberto porque é ambiente de demonstração; em produção restringiríamos à origem do app.

Como o upload de foto funciona?

O app manda a imagem como multipart/form-data; o multer grava o arquivo em backend/uploads com nome aleatório (limite de 8 MB) e a rota guarda no banco só o caminho. O Express serve a pasta como arquivos estáticos em /uploads.

A IA é real?

Não. O contrato da API já está pronto (recebe a foto e devolve praga, confiança, área afetada e recomendação), mas a análise é simulada no servidor: sorteia uma praga do catálogo. O próximo passo é trocar esse trecho por um modelo de visão computacional treinado, sem mudar o app.

Como evita SQL injection?

Usando prepared statements: o SQL fica fixo com parâmetros (?), e os valores do usuário são passados separadamente, nunca concatenados.

O que é ON DELETE CASCADE?

Regra da chave estrangeira: ao apagar o registro pai (usuário), o banco apaga sozinho os filhos (setores, análises, ordens). Evita registros órfãos.

Como você testou?

Testes manuais no celular e um teste automático de ponta a ponta (npm run smoke) com 51 verificações que exercitam cadastro, login, erros 400/401/404/409, upload, mudança de status e isolamento entre usuários. Também validamos num clone limpo do GitHub.

E se o servidor cair ou não houver internet?

O app mostra uma mensagem de erro clara e desiste em 15 segundos, em vez de travar. Ainda não há fila offline: o selo OFFLINE/QUEUED da interface é simulado. Fila local com sincronização é o próximo passo (a proposta da startup é justamente funcionar offline em campo).

Como isso escalaria?

PostgreSQL no lugar do SQLite, backend hospedado na nuvem com HTTPS, fotos em armazenamento de objetos (ex: S3), segredo do JWT em variável de ambiente, refresh token, limite de tentativas de login e o modelo de IA rodando no aparelho ou em um serviço próprio.

10. Limitações (fale antes que perguntem — passa maturidade)

Limitação atual	Como resolveria
A análise de IA é simulada no servidor.	Trocar o sorteio por um modelo real (TensorFlow Lite / servidor de inferência).
O segredo do JWT tem um valor padrão no código (o repositório é público).	Definir `JWT_SECRET` forte no `.env` do servidor. Aqui o backend só roda em rede local, para demonstração.
Tráfego em HTTP (sem HTTPS).	Certificado TLS / proxy reverso em produção.
CORS aberto e sem limite de tentativas de login.	Restringir a origem e adicionar rate limit.
Servidor só exige senha não vazia (o mínimo de 4 caracteres é validado no app).	Validar tamanho/complexidade também no servidor.
O upload aceita qualquer tipo de arquivo (só limita o tamanho).	Validar o tipo (imagem) e redimensionar.
Token de 30 dias, sem revogação, guardado sem criptografia.	Token curto + refresh token + armazenamento seguro.
Sem testes unitários (há o teste de ponta a ponta).	Adicionar testes de rota com Jest/Supertest.
Um único arquivo de banco / um servidor.	PostgreSQL e múltiplas instâncias.

11. Números para ter na ponta da língua

Item	Valor	Item	Valor
Porta da API	4000 (escuta em 0.0.0.0)	Endpoints	13 (+ arquivos em /uploads)
Validade do JWT	30 dias (algoritmo HS256)	bcrypt	custo 10
Tabelas	5 (users, pests, sectors, detections, service_orders)	Limite de upload	8 MB por foto
Pragas no catálogo	6	Timeout do app	15 s (JSON)
Cadastro cria	4 setores + 3 ordens + 3 análises	Testes automáticos	51 verificações (npm run smoke)
Backend	Node 20/22/24 · Express 5	Banco	SQLite via better-sqlite3 12.11.1
Comandos	`npm run dev` · `npm run db` · `npm run smoke`	Arquivo do banco	`backend/data/agroscan.db`